

Week 04 — Detailed Task Breakdown

1. Palak Agarwal — Student Domain

Day 01 — Student Status

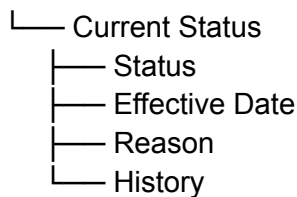
Goal: Manage the current status/state of a student.

Implement:

- Student status master
 - Active
 - Inactive
 - Suspended
 - Graduated
 - Dropped
 - Transferred
 - Alumni
- Change student status
- Status history
- Effective date of status
- Reason for status change
- Status validation
- API endpoints:
 - Get current status
 - Change status
 - Get status history
- Prevent invalid transitions where applicable
- Test APIs

Expected output:

Student



Day 02 — Student APIs & Testing

Goal: Make the Student module properly accessible through APIs and test it.

Implement/test:

- Student CRUD APIs
- Get student by ID
- Get student by enrollment/registration number
- Search students
- Filter students
- Pagination
- Student status API

- Guardian-related APIs if already implemented
- Document-related APIs if already implemented
- Validation/error responses
- HTTP status codes

Testing:

POST

GET

PUT/PATCH

DELETE

Test:

- Valid request
- Invalid ID
- Missing fields
- Duplicate enrollment number
- Unauthorized request
- Invalid status
- Empty search
- Pagination

Deliverable: Postman collection + API test evidence.

Day 03 — Student Enrollment

Goal: Connect a student with an academic structure.

Implement:

Student

↓

Enrollment

↓

Academic Year

↓

Program

↓

Batch

↓

Semester

Features:

- Student enrollment
- Enrollment number
- Academic year
- Program
- Batch
- Semester
- Enrollment date
- Enrollment status
- Re-enrollment
- Transfer between academic structures where applicable

- Prevent duplicate enrollment
- Enrollment history

Example:

Lakshya

→ B.Tech AI&DS

→ 2026–27

→ Batch A

→ Semester 5

→ Active

Day 04 — Student Academic Records

Goal: Maintain the student's academic information.

Implement:

- Student academic history
- Semester-wise records
- Course/subject enrollment
- Marks/results reference
- GPA/CGPA reference
- Academic year
- Semester
- Academic status
- Previous academic records

Important:

This should **not duplicate the Examination module**. Student should consume/reference examination results rather than creating another marks system.

Day 05 — Student Category & Classification

Goal: Classify students for institutional purposes.

Implement category/classification such as:

Category

- General
- OBC
- SC
- ST
- Other

And potentially institutional classifications supported by your SRS:

- Student type
- Admission category
- Scholarship category
- Special category
- Minority/category flags where applicable

Requirements:

- Category master
- Assign category to student
- Update category
- Search/filter by category
- Validation
- API

Avoid hard-coding categories into Java code.

Day 06 — Student Lifecycle Workflow

This is the **biggest Student-domain task** of the week.

Model:

Applicant



Admitted



Enrolled



Active Student



Suspended / Transferred / Dropped



Graduated



Alumni

Implement:

- Lifecycle states
- Valid transitions
- Transition history
- Transition date
- Reason
- Actor/user who performed transition
- Validation
- APIs
- Audit trail

Example:

ACTIVE → GRADUATED

ACTIVE → SUSPENDED

ACTIVE → TRANSFERRED

ACTIVE → DROPPED

GRADUATED → ALUMNI

2. Manish Sharma — Security

Day 01 — Session Management

Implement backend session management.

Tasks:

- Login session creation
- Session ID/token tracking
- Session expiry
- Logout
- Session invalidation
- Multiple-session handling
- Session information
- Last activity
- Session timeout
- Session revocation

For JWT architecture, clearly define what is handled by:

Access Token

Refresh Token

Session

Do not simply create another unnecessary session system on top of JWT.

Day 02 — Authentication Testing & Integration

Integrate and test authentication with the rest of the backend.

Test:

Login

↓

JWT/access token

↓

Authenticated API

↓

Authorization

Test cases:

- Valid login
- Invalid password
- Invalid username
- Missing token
- Expired token
- Malformed token
- Logout
- Refresh token
- Unauthorized endpoint
- Concurrent sessions

Also integrate authentication with Student/Academic APIs.

Day 03 — User Management

Create the central system-user management functionality.

Implement:

- Create user
- Update user
- Deactivate user
- Activate user
- Get user
- Search users
- User status
- User profile
- Username/email uniqueness
- Password handling
- User audit information

Example:

User

```
|— ID
|— Username
|— Email
|— Password Hash
|— Status
|— Created At
|— Updated At
```

Day 04 — User–Role Assignment

Implement:

User

↓

Role

↓

Permissions

Features:

- Assign role
- Remove role
- Multiple roles
- View assigned roles
- Role validation
- Role assignment history if required
- API endpoints

Example:

Manish

```
|— SECURITY_ADMIN
|— AUDITOR
```

Day 05 — Permission Enforcement

This is different from merely creating roles.

Actually enforce permissions.

Example:

STUDENT_CREATE
STUDENT_READ
STUDENT_UPDATE
STUDENT_DELETE

Then:

POST /students



Authentication



User



Role



Permission



ALLOW / DENY

Implement:

- Endpoint authorization
- Permission checks
- Role-based access
- 401 handling
- 403 handling
- Method-level authorization where appropriate
- Test unauthorized access

Day 06 — Password & Account Recovery

Implement:

Password

- Change password
- Password validation
- Password hashing
- Old-password verification
- Password policy

Recovery

- Forgot password
- Reset password
- Reset token

- Token expiry
- One-time token usage
- Account recovery validation

Security requirements:

- Never store plain-text passwords
 - Never return passwords through APIs
 - Expire recovery tokens
 - Prevent token reuse
-

3. Lakshya Singhal — Core Frontend

His Week 04 is essentially the **frontend foundation + reusable architecture**.

Day 01 — Advanced Data Management

Build the frontend's reusable data-management layer.

Implement:

- API service structure
- API client
- Request/response handling
- Loading state
- Error state
- Success state
- Pagination state
- Filtering state
- Sorting state
- Search state
- Data caching/state management where applicable

Architecture should look roughly like:

Component



Hook



Service/API



Backend

Not:

Component



fetch() everywhere

Day 02 — Frontend–Backend Integration Architecture

Define how the entire frontend communicates with Spring Boot.

Implement:

React



API Client



HTTP



Spring Boot

Need:

- Base API URL
- Axios/fetch abstraction
- Request interceptors
- Response interceptors
- Authentication token handling
- Error handling
- API service organization
- Environment configuration
- DTO/type definitions
- Standard API response handling

Example:

```
src/  
├── services/  
│   ├── apiClient.ts  
│   ├── auth.service.ts  
│   ├── student.service.ts  
│   └── institution.service.ts  
├── hooks/  
└── types/
```

This becomes the foundation for every later frontend module.

Day 03 — Student Module Frontend

Build actual UI for Palak's Student APIs.

Screens/components:

Student Dashboard

Student List

Student Details

Create Student

Edit Student

Student Status

Student Academic Information

Student list should support:

- Search
- Filter
- Pagination
- Sorting
- View
- Edit
- Status display

Integrate with the real backend APIs.

No hard-coded fake student data for the final implementation.

Day 04 — Institution Module Frontend

Build UI for:

Institution

Campus

Department

Possible screens:

Institution List

Institution Details

Campus Management

Department Management

Implement:

- CRUD UI
 - Forms
 - Tables
 - Search
 - Filters
 - Pagination
 - API integration
 - Loading states
 - Empty states
 - Error states
-

Day 05 — Advanced Forms & Dynamic Validation

Build reusable forms for the ERP.

Need:

- Text input
- Number input
- Email
- Date
- Select
- Multi-select
- Checkbox
- Radio
- Required validation
- Format validation
- Min/max validation
- Field-level errors
- Form-level errors
- Submit/loading state

Dynamic validation

Example:

Student Type = International



Passport Number becomes required

Another:

Program = B.Tech



Show Engineering-specific fields

The goal is a **reusable form system**, not validation written separately inside every component.

Day 06 — Advanced DataTable & Server-Side Operations

Create a **reusable ERP DataTable**.

Features:

- Server-side pagination
- Page size
- Server-side sorting
- Search
- Filtering
- Column configuration
- Row actions
- View
- Edit
- Delete
- Loading state
- Empty state

- Error state

Example:

Search Student	Filter ▼	
Name	Enrollment	Program Status ...
...		
← 1 2 3 4 5 →	20 / page	

This DataTable should later be reusable for:

Students
Faculty
Courses
Subjects
Fees
Attendance
Examinations
Users

4. Prashant Jain — Academic Domain

Day 01 — Semester

Implement Semester management.

Features:

- Create semester
- Update semester
- Delete/deactivate semester
- Semester number/name
- Academic year association
- Start date
- End date
- Status
- Validation

Example:

Academic Year: 2026–27

- Semester 1
- Semester 2
- Semester 3
- Semester 4

Day 02 — Program & Batch Management

Program

Implement:

- Create program
- Update
- Activate/deactivate
- Program code
- Program name
- Degree
- Department
- Duration

Example:

B.Tech AI&DS

B.Tech CSE

BCA

MBA

Batch

Implement:

- Batch creation
- Program association
- Admission year
- Graduation year
- Batch code
- Status

Example:

B.Tech AI&DS

↓

2026 Batch

↓

Section A / B

Day 03 — Course Management

Implement Course master.

Fields could include:

Course Code

Course Name

Course Type

Credits

Department

Program

Status

Features:

- CRUD

- Search
 - Filter
 - Activate/deactivate
 - Course code uniqueness
-

Day 04 — Curriculum Management

This connects academic structures together.

Example:

Program



Semester



Curriculum



Courses/Subjects

Implement:

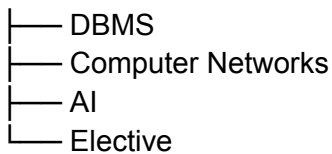
- Curriculum creation
- Assign courses
- Semester mapping
- Credit structure
- Mandatory/elective classification
- Curriculum version
- Effective academic year
- Validation

Example:

B.Tech AI&DS

Semester 5

2026 Curriculum



Day 05 — Subject Management

Implement Subject master.

Features:

- Subject code
- Subject name
- Subject type
- Credits
- Theory/practical
- Department
- Course/program mapping

- Status
- Search/filter

Potential distinction:

Course



Subject

But the exact relationship should follow your SRS/domain model; don't create duplicate concepts unnecessarily.

Day 06 — Section/Class Management

Implement academic sections/classes.

Example:

B.Tech AI&DS



2026 Batch



Semester 5



Section A

Features:

- Create section
- Assign batch
- Assign semester
- Section capacity
- Section status
- Student assignment
- Faculty/class association where applicable
- Search/filter
- Validation